இனக்குழு உறுப்புகள்	Private காண்புநிலை பாங்குடன் – (அடிப்படை இனக்குழுவின் பண்புக்கூறுகளை பெறுதல்.		
	Private	Protected	Public
shape (அடிப்படை இனக்குழு)	int count;	int width; int height;	voidsetWidth(int) voidsetHeight(int)
rectangle (அடிப்படை இனக்குழுவின் பண்புக்கூறுகளை private காண்புநிலையுடன் தருவிக்கப்பட்ட இனக்குழு பெறுகிறது.)	அடிப்படை இனக்குழுவின் private உறுப்புகளை தருவிக்கப்பட்ட இனக்குழு நேரடியாக அணுக முடியாது.	int width; int height; void setWidth(int) void setHeight(int)	intgetArea();

ஏற்கனவே இருக்கும் இனக்குழுவின் அடிப்படையில் புதிய இனக்குழுவை தருவிக்கும் போது, காண்பு நிலை பாங்கின் அடிப்படையில் தருவிக்கப்பட்ட இனக்குழு அடிப்படை இனக்குழுவின் பண்புகளை மரபு வழி பெறுகிறது. எனவே, தேவைக்கேற்றார் போல், உரிய காண்பு நிலை பாங்கினை பயன்படுத்த வேண்டும்.

private என வகைப்படுத்தப்பட்ட அடிப்படை இனக்குழுவின் பண்புக்கூறுகளை தருவிக்கப்பட்ட இனக்குழுவின் உறுப்புகள் மட்டுமே பயன்படுத்திக்கொள்ள முடியும்.

ஆனால் அடிப்படை இனக்குழுவின் பண்புக்கூறுகள் தருவிக்கப்பட்ட இனக்குழுவின் அடிப்படையில் உருவாக்கப்பட்ட இனக்குழுவிற்கு இது பொருந்தாது.

protected என வகைப்படுத்தப்பட்ட தருவிக்கப்பட்டஇனக்குழுவின் பண்புக்கூறுகளை தருவிக்கப்பட்ட இனக்குழுவின் உறுப்புகள் மட்டுமே எடுத்தாள முடியும். இனக்குழுவிற்கு வெளியே அதனை எடுத்தாள முடியாது.

public என வகைப்படுத்தப்பட்ட அடிப்படை இனக்குழுவின் பண்புக்கூறுகளை தருவிக்கப்பட்ட இனக்குழு மட்டுமல்லாது, இனக்குழுவுக்கு வெளியேயும் எடுத்தாள முடியும்.

16.6 மரபுரிமம், ஆக்கிகள் மற்றும் அழிப்பிகள்

ஒரு தருவிக்கப்பட்ட இனக்குழுவின் பொருளை உருவாக்கும் போது, நிரல் பெயர்ப்பி முதலில் அடிப்படை இனக்குழுவின் ஆக்கியை அழைக்கும், அதன்பின் தருவிக்கப்பட்ட இனக்குழுவின் ஆக்கியை அழைக்கும். ஏனெனில், தருவிக்கப்பட்ட இனக்குழு உறுப்புகள் அடிப்படை இனக்குழுவின் மீது கட்டமைக்கப்பட்டுள்ளது. தருவிக்கப்பட்ட இனக்குழுவின் பொருளின் பயன்பாடு முடிவுக்கு வரும்போது தருவிக்கப்பட்ட இனக்குழுவின் அழிப்பி முதலில் செயல்படுத்தப்பட்டு, அடுத்து அடிப்படை இனக்குழுவின் அழிப்பி இயக்கப்படும்.

எடுத்துக்காட்டு நிரல் 16.4 ஆக்கிகள், அழிப்பிகள் வரிசைமுறையை விளக்கிக்காட்டுகிறது.

```
#include<iostream>
using namespace std;
class base
{
public:
base()
{
cout<<"\nConstructor of base class...";
}
~base()
{
cout<<"\nDestructor of base class.... ";
}
};
class derived:public base
{
public :
derived()
{
cout << "\nConstructor of derived ...";
}
~derived()
{
cout << "\nDestructor of derived ...";
}
};
class derived1 :public derived
```


Chapter 16.indd 269 4/19/2020 4:15:41 PM